

LOGIC & CONTROL FLOW MASTERY

Booleans, Conditionals, and Loops

CodeHive Academy

Module 8: The Engine of Decision Making

1. C++ Booleans: The Foundation of Logic

Booleans allow your program to differentiate between what is **true** and what is **false**. Without this, software could not make choices.

Boolean Essentials

- **The bool Keyword:** Used to declare a variable that can ONLY be true or false.
- **Memory Usage:** Typically occupies 1 byte of memory.

Code Implementation:

```
bool isPlatformLive = true; bool maintenanceMode = false;
```

2. Boolean Expressions

An expression that returns a boolean value is the core of most conditional checks.

```
int x = 10; int y = 9; cout << (x > y); // Outputs 1 (true) cout << (x == 10); // Outputs 1 (true)
```

Real-Life Scenario: Access Control

Checking if a CodeHive user has enough balance to purchase a specific course module:

```
int userBalance = 50; int courseCost = 40; bool canPurchase = (userBalance >= courseCost); // Result: true
```

3. C++ If...Else: Making Decisions

Conditional statements tell your program which path to take based on the logic evaluated above.

- **if:** The initial check.
- **else if:** A secondary check performed only if the first check fails.
- **else:** The final fallback if everything else is false.

```
int score = 85; if (score ≥ 90) { cout << "Grade: A"; } else if (score ≥ 80) { cout << "Grade: B"; // This logic executes } else { cout << "Keep trying!"; }
```

4. Efficiency: Short Hand & Logic

Short Hand If (Ternary Operator)

Replaces a simple if-else with a single line of code, perfect for direct assignments.

```
// Syntax: variable = (condition) ? trueValue : falseValue; string status =  
(score > 50) ? "Pass" : "Fail";
```

Complex Logical Operators

- **&& (AND):** True only if **both** sides are true.
- **|| (OR):** True if **at least one** side is true.

```
if (isLoggedIn && isAdmin) { cout << "Welcome to the Admin Dashboard"; }
```

5. C++ Switch: Multi-Way Branching

Use `switch` to select one of many code blocks to execute based on a single variable's discrete value.

- **case:** A specific value to match against the switch variable.
- **break:** Stops the code from "falling through" to the next case in line.
- **default:** The safety net code that runs if no case matches.

```
char grade = 'B'; switch (grade) { case 'A': cout << "Excellent!"; break;  
case 'B': cout << "Very Good!"; break; // Executes default: cout << "Invalid  
Entry"; }
```

6. C++ While Loop: Repeating Tasks

Loops keep your code running efficiently as long as a condition remains true. This is fundamental for data processing.

Condition-First Performance

The while loop checks the condition **before** running any code inside.

```
int i = 0; while (i < 3) { cout << i << " "; i++; // Crucial: Increments to prevent infinite loops }
```

Academy Note: If you forget to increment your counter, the condition never becomes false, causing a **Hang/Crash**.

7. Ensuring Execution: Do/While

Unlike the standard while, the **do/while** loop executes the code block once before checking the condition.

```
int j = 0; do { cout << j << " "; j++; } while (j < 3);
```

Comparison Summary

While: Best when you might not need to run the code at all.

Do/While: Best for user interfaces where you must show the screen at least once.

8. Logic & Loop Conclusion

You have now explored the primary methods C++ uses to handle automated decision-making and data repetition.

CodeHive Best Practice:

- Use `bool` for flag states (On/Off).
- Prefer `switch` for exact matches (Menus/Grades).
- Ensure loops have a definitive **Exit Condition**.

Code. Learn. Repeat.

© 2026 CodeHive Academy. Logic, Control, & Iteration Guide.